GUI-Avanceret GUI-programmering

Indhold

Faglige mål 2
Målpind som dagens øvelse dækker 2
Baggrund 3
Forstår din applikations liftetime-events, brug af asynkrone og custom controls 3
Øvelse 4
1. Forstå din applikation lifetime-events 4
2. Custom controls 4
Hvordan du bliver bedømmet 4

Faglige mål

Målpind som dagens øvelse dækker

1. **Eleven kan foretage GUI-programmering rettet mod både clientside-applikationer, og webbaserede applikationer, der bygger på SPA (Single Page Application)**
2. **Eleven kan implementere en grafisk brugergrænseflade, som understøtter et eller flere funktionelle krav.**
3. **Eleven kan anvende forskellige typer af layout containers.**
4. **Eleven kan udvælge og konfigurere de korrekte kontroller til at understøtte et eller flere funktionelle krav.**
5. **Eleven kan implementere "event-handling".**
6. **Eleven kan implementere et UI Design pattern.**
7. ***Eleven kan designe, udvikle og anvende "custom-controls".**
8. ***Eleven kan konfigurere Application-klassen og dens lifetime-events.**
9. ***Eleven kan implementere en multitrådet applikation.**
10. *Eleven kan udføre en simpel usability-test.*

Baggrund

Forstår din applikations liftetime-events, brug af asynkrone og custom controls

I moderne GUI-applikationer er bruget af **lifetime-events**, **asynkrone** og **Custom Controls** vigtigt til:

- tilpas/ændre applikations GUI efter ændring i applikations tilstand.
- hente og behandle data uden at blokere brugergrænsefladen
- vise en midlertidig status, mens data indlæses
- genbruge den samme funktionalitet flere steder i applikationen
- skabe en mere responsiv og brugervenlig GUI

Øvelse

1. Forstå din applikation lifetime-events

Du skal kunne *konfigurere Application-klassen og dens lifetime-events*. Din **Application-klassen er et centralt objekt, som repræsenterer selve applikationen**, og at du kan koble kode på bestemte tidspunkter i applikationens levetid.

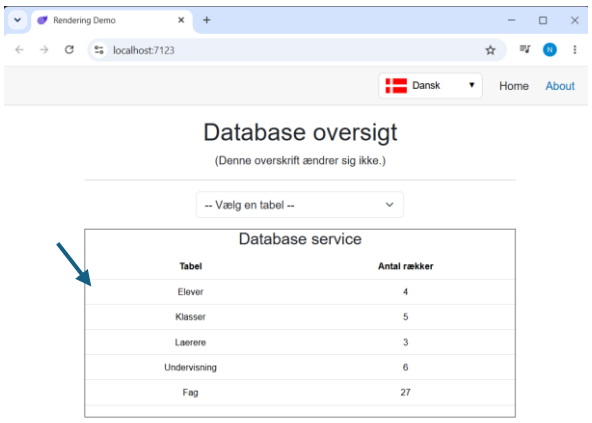
I **Blazor** vil man typisk gøre dette ved at bruge komponenternes lifecycle (**OnInitialized**, **OnParametersSet**, **OnAfterRender**, **Dispose**) og/eller eget implementeret services/events.

2. Custom controls

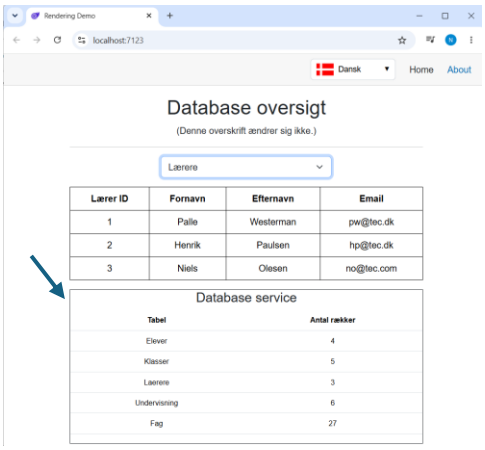
1.1. Du skal implementer en "custom control" til din SPA applikation:

- Din custom control skal funger som en **database service** der:
  - Køre som en baggrundprocess, og loades når applikation startes.
  - Det skal være **live data** hentet fra din database
    - Viser hvor mange tabeller der findes i din database og hvor mange rækker der findes for hver eksisterende table.
    - Den skal automatisk opdater sin viste information (f.ex. ved ændringer i antal række fordi du insæt eller sletter en record, opdater visning live).
  - Du skal selv bestemme det visuelle design.

Eks: fra SkoleDb:



Stadig vises når en table vælges



1.2. **Reflektér over din løsning.**

Du skal kunne vise hvor og hvordan du i din kode har implementeret løsning til asyncron kørsel, din håndtering af applikations lifetime-events af din live data visning i din custom control, men også kan forklare begreberne brugt i forbindelse med GUI nævnt og behandlet under forløbet. Vurdering anvendes som en del af det samlede vudering ved karaktergivning.

Hvordan du bliver bedømmet

- Du skal fremvis kørsel af din application der opfylder de stillede krav.
- Opgave indeholder en "refleksion" øvelse.
  - Du bliver bedømmet via spørgsmåler stillet med udgangspunkt i refleksionen af din øvelser.